UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Centralized Parking System

Name: Florea Deborah Ruxandra Maria
Group: 30435

Table of Contents

Deliverable 1

Project Specification

The centralized parking system is a web-based application designed to collect and organize information about all public parking spots in a city and display it on a unified platform.

The system provides real-time updates regarding parking availability, pricing, and location, allowing users to quickly identify suitable parking options. It integrates with external navigation services (Google Maps) to guide users to their selected parking location and provide additional information like reviews and busyness.

The main objective is to reduce the time spent searching for parking, improve traffic flow, and decrease carbon emissions.

Functional Requirements

The system must provide the following functionalities:

- Users (drivers) can view a list of parking lots with real-time availability
- Users can search and filter parking lots by name, location, price, or availability
- Drivers can navigate to a selected parking lot using Google Maps
- Drivers can keep track of their bills: how much they spend on parking
- Parking Lot Managers can update free/taken parking spaces in real time
- Parking Lot Managers can update pricing and payment options
- Parking Lot Managers can view statistics for their parking lot
- Administrators can add or remove parking lots
- Administrators can assign parking lot managers
- The system performs CRUD operations on parking data and user interactions

Use Case Model

Use Cases Identification:

Use-Case 1: View Parking Lots

- **Level:** User goal
- **Primary Actor:** Driver

Main success scenario:

1. Driver logs into the system.
2. System displays available parking lots.
3. Driver views details (availability, price, location).

Extensions:

- If no parking is available → system displays notification
- If user is not logged in → redirect to login page

Use-Case 2: Update Parking Availability

- **Level:** Sub-function
- **Primary Actor:** Parking Lot Manager

Main success scenario:

1. Manager logs in

2. Updates number of free/taken spots
3. System updates database in real time

Extensions:

- Invalid data → system rejects update
- Connection error → retry update

Use-Case 3: Add Parking Lot

- **Level:** Administrative
- **Primary Actor:** Administrator

Main success scenario:

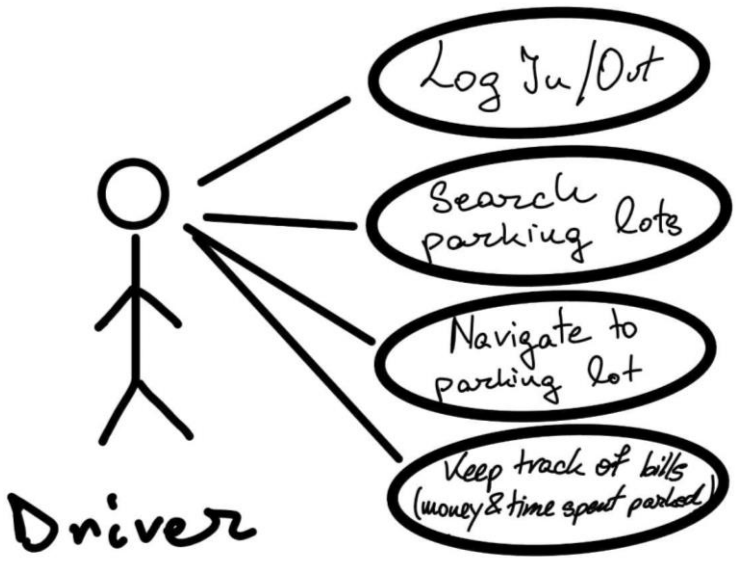
1. Administrator logs in
2. Enters new parking lot details
3. Assigns manager
4. System stores data

Extensions:

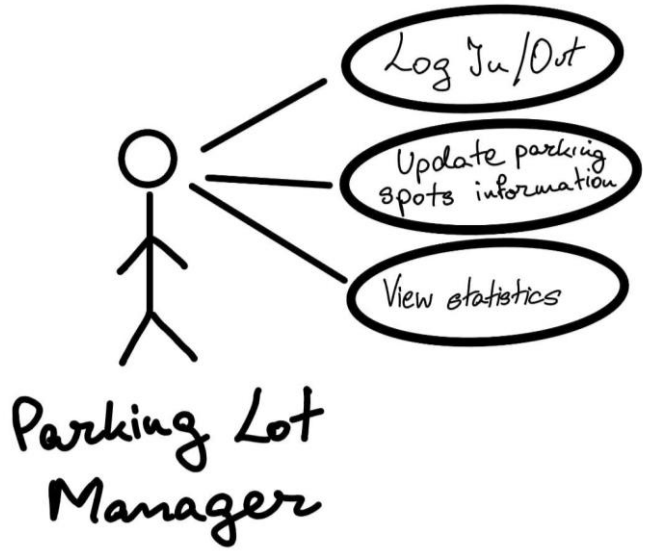
- Missing data → system requests completion
- Duplicate parking lot → system rejects entry

UML Use Case Diagrams

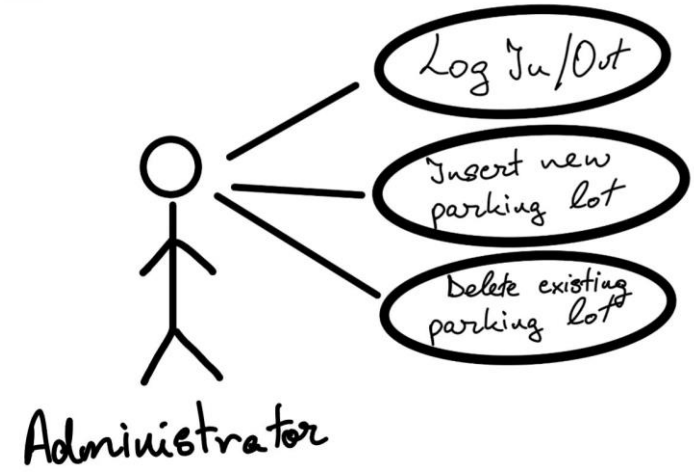
- Use Case Diagram for *Driver*



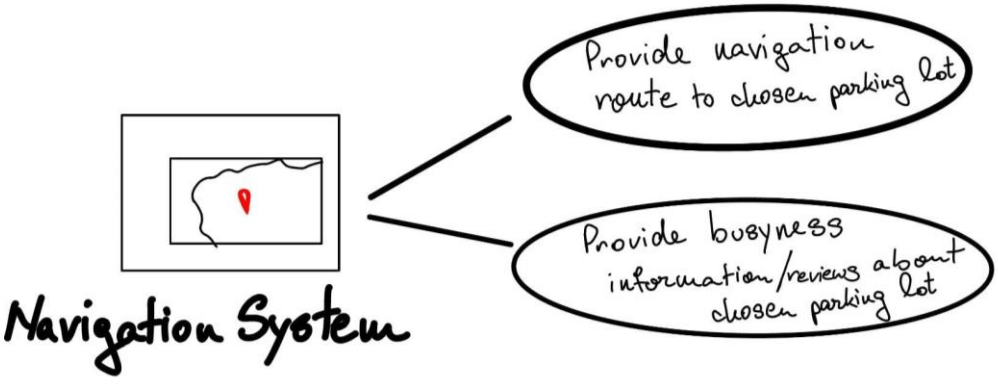
- Use Case Diagram for *Parking Lot Manager*



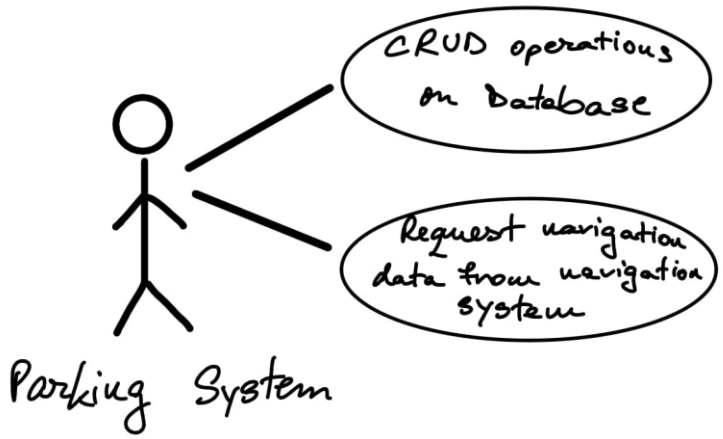
- Use Case Diagram for *Administrator*



- Use Case Diagram for *Navigation System*



- Use Case Diagram for *Parking System*



Supplementary Specification

Non-functional Requirements

1. Performance: The system must provide real-time updates with minimal delay to ensure accurate parking availability.
2. Usability: The interface must be intuitive and accessible from multiple devices (desktop, tablet, mobile).
3. Scalability: The system must support a growing number of users and parking lots using a client-server architecture.
4. Reliability: The system must ensure consistent data synchronization between users and database.

Design Constraints

- Backend must use Java + Spring Boot
- MVC architecture must be implemented
- Database access via Spring Data JPA
- Integration with Google Maps API
- Web-based interface using HTML, CSS, Bootstrap, Thymeleaf

Glossary

Term	Use cases	Description
Administrator	Administrator Log-in / Log-out, Entering a new parking lot into the Database, Deleting a parking lot from the Database, Checking reviews	The person responsible for maintaining the system, managing database entries, and moderating reviews.
Driver	Driver Log-in / Log-out, Driver Page: List of parking lots, Leave reviews for visited parking lots, Navigate to parking lot	The user searching for parking information and navigating to available parking spaces.
Parking Lot Manager	Parking Lot Manager Log-in / Log-out, Editing the number of free/taken parking spaces, Respond to reviews from visitors	The user who manages real-time parking data, pricing, and feedback for a specific parking lot.

Navigation System	Navigate to parking lot, Busyness of the parking lot during a specific hour	The external service (Google Maps) that provides directions, traffic data, and congestion insights.
Parking Lot	All use cases related to viewing, editing, or navigating to parking spaces	A designated area where vehicles can be parked. Each parking lot has specific attributes such as location, capacity, pricing, payment options and an assigned Parking Lot Manager.
Parking Spot	Editing the number of free/taken parking spaces, View parking availability	A single vehicle space within a parking lot that can be occupied or free.
Parking System	Insert Parking Lot/ Delete Parking Lot/ Update Availability/ Update Pricing/ Retrieve Parking Lot List/ Search Parking Lots/ Request Routing Information	Central software component that performs all CRUD operations on the database and communicates with the Navigation System to fetch routing and busyness information, ensuring accurate real-time data for all actors.

Deliverable 2

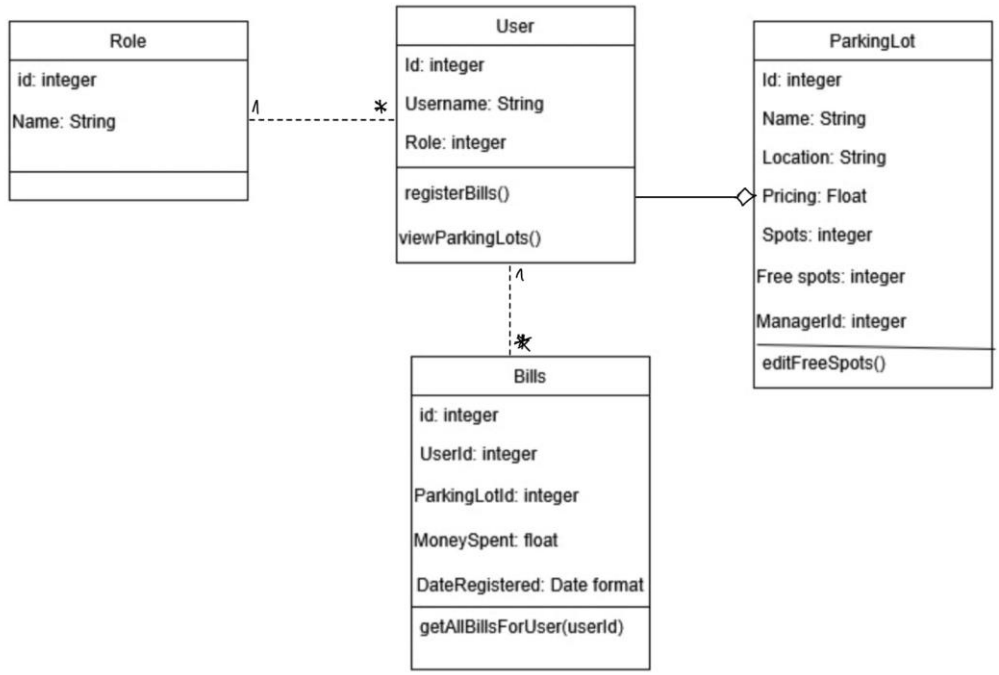
Domain Model

The domain model includes the following main entities:

- User
- ParkingLot
- Role (Administrator, Driver, Manager)
- Bills

Relationships:

- Many Users can have the same role
- A ParkingLot cannot exist without its one Manager
- A Driver can have multiple Bills



Architectural Design

Conceptual Architecture

The application uses a Client-Server architecture, while the system utilizes the Model-View-Controller (MVC) architectural pattern facilitated by the Spring Boot framework.

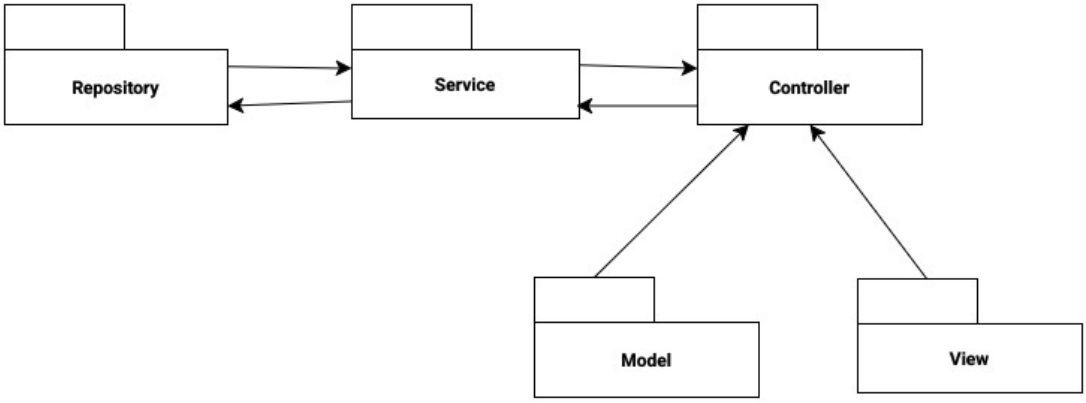
The system's conceptual architecture is divided into four distinct layers:

1. Presentation Layer (Client/UI):
    - This layer is built using HTML, CSS, Bootstrap, and Thymeleaf.
    - It is responsible for displaying real-time parking information and interacting with the system's users.
    - It acts as the "View" in the MVC pattern, rendering dynamic, personalized HTML content based on the user's role.
2. Application Layer (Server Logic):
    - This layer contains the system's controllers, services, and business rules.
    - It is built using Java and Spring Boot.
    - It handles core operations including authentication, authorization, CRUD (Create, Read, Update, Delete) operations, and processing real-time updates.
    - It acts as the "Controller" in the MVC pattern, managing HTTP requests (GET, POST, PUT, DELETE) and coordinating data between the View and the Model.
3. Data Layer:
    - This layer utilizes Spring Data JPA to communicate with a relational database.
    - It stores crucial domain entities (the "Model" in MVC) such as users, parking lots, pricing, and real-time space availability.
4. External Integration Layer:
    - This layer manages communication with external services, specifically the Google Maps API.
    - It is responsible for generating directions, fetching location data, and retrieving predicted busyness information.

Motivation and Choice of Architecture

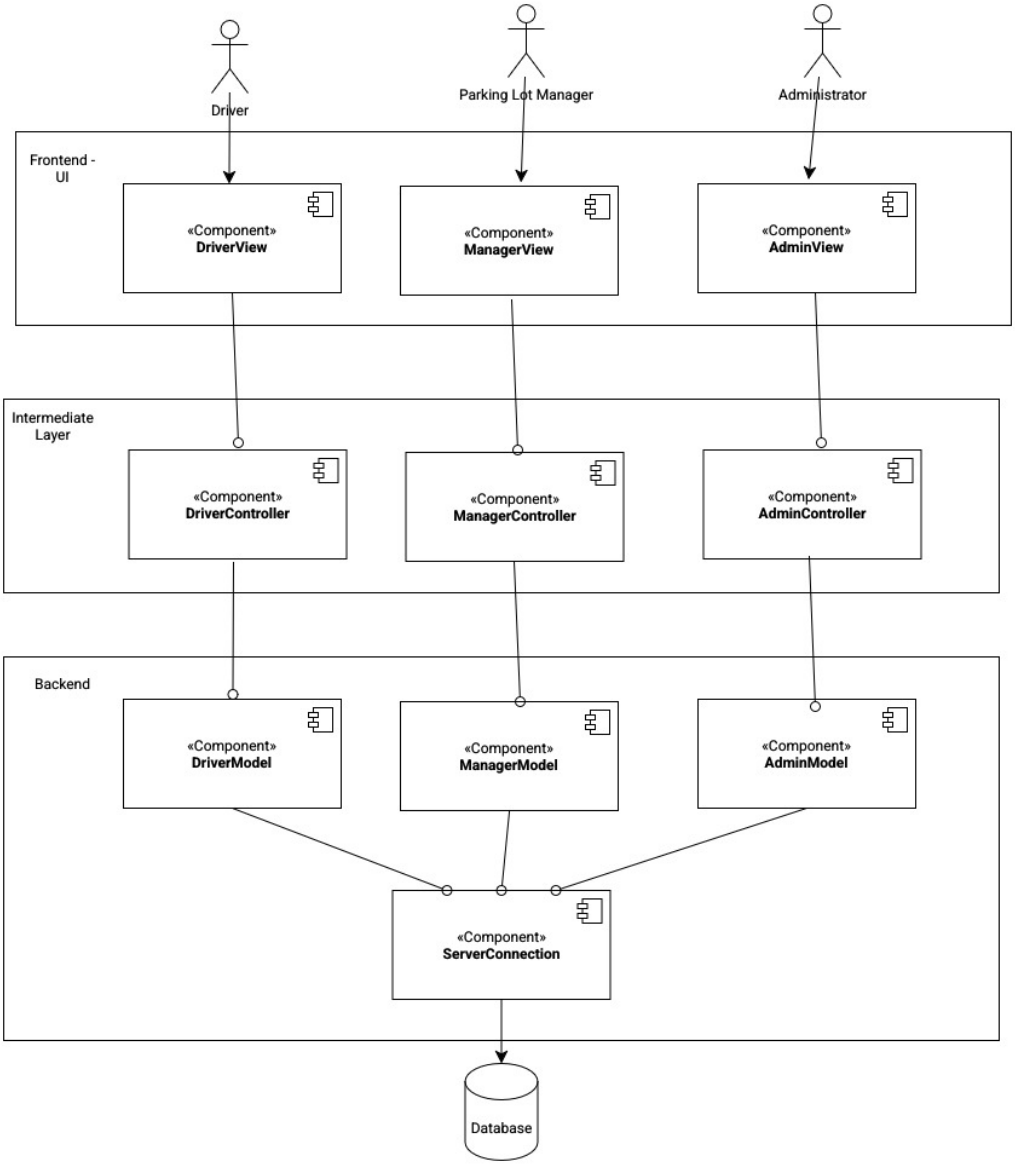
- Why Client-Server?: A client-server architecture with centralized server management was chosen because it inherently supports scalability, ensures data consistency and allows for efficient data distribution among multiple clients accessing shared information.
- Why MVC (via Spring Boot)?: The MVC pattern maps entity classes to relational database tables (Model), generates dynamic user interfaces (View), and coordinates logic and HTTP requests (Controller). Combined with Spring Data JPA for abstracting CRUD operations, this architectural choice creates a maintainable backend capable of supporting real-time interactions and secure authentication.

Package Design

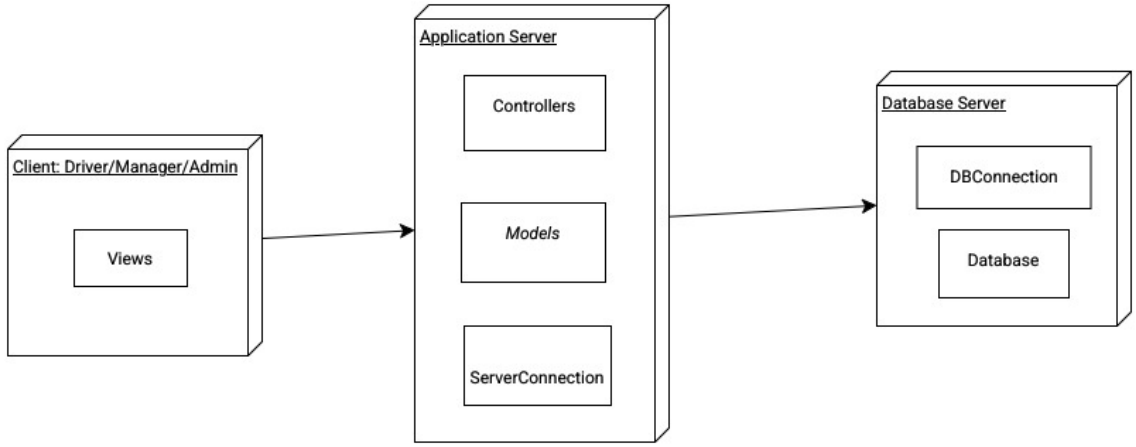


Component and Deployment Diagram

- Component diagram



- Deployment diagram



Deliverable 3

Design Model

Dynamic Behavior

[Create the interaction diagrams (2 sequence) for 2 relevant scenarios]

Class Diagram

[Create the UML class diagram; apply GoF patterns and motivate your choice]

Data Model

[Create the data model for the system.]

System Testing

[Describe the testing methides and some test cases.]

Future Improvements

Conclusion

Bibliography

[1] Spring, "Spring Boot documentation." [Online], https://docs.spring.io/spring-boot/documentation.html
[2] Google, "Google Maps Platform documentation." [Online], https://developers.google.com/maps/documentation
[3] GeeksforGeeks, "Spring Boot architecture." [Online], https://www.geeksforgeeks.org/springboot/spring-boot-architecture/
[4] DoiT International, "Design and development resources for the Google Maps Platform." [Online], https://www.doit.com/blog/design-and-development-resources-for-the-google-maps-platform/

[5] U. Mahajan, “Mastering software design patterns in Java with Spring Boot.” [Online], https://medium.com/@umeshmahajan8297/mastering-software-design-patterns-in-java-with-spring-boot-a-practical-guide-7d97999bc5ca
[6] GeeksforGeeks, “Spring Boot JpaRepository with Example.” [Online], https://www.geeksforgeeks.org/springboot/spring-boot-jparepository-with-example/